TECHNICAL REPORT: INTEGRATED SECURITY ASSESSMENT, PYTHON TOOLKIT, AND FIREWALL CONFIGURATION

Module: ETTCS801 Cryptography and Network Security

Peace MUNYANEZA
ULK Polytechnic Institute

September 29, 2026

Abstract

This technical report documents the implementation of security measures for a polytechnic institute's central server and campus network. It covers a comprehensive risk assessment, a custom Python cryptography and file integrity toolkit, iptables network traffic filtering, verification test cases, and repository deployment instructions.

Contents

1 Introduction **2**

2 Risk Assessment **2**
2.1 Assets, Vulnerabilities, and Consequences 2
2.2 Risk Rankings and Justification 2
2.3 Recommended Controls 3

3 Encryption and Integrity Application **3**
3.1 Implementation Overview 3
3.2 Core Cryptographic Implementation 3
3.3 Verification Procedures 4

4 Network Traffic Filtering **4**
4.1 Firewall Rule Definitions 4
4.2 Filter Verification Test Execution 5

5 Project Evidence and GitHub Repository Submission **5**
5.1 Repository Layout 6

6 Conclusion **6**

1 Introduction

A recent security review identified multiple vulnerabilities across the institution's network, including unencrypted student record file transfers, weak staff passwords, outdated software, and unencrypted guest network access to the primary records server. Additionally, repetitive unauthorized external access attempts were observed. This project implements risk mitigation controls, an automated file encryption and integrity checking toolkit in Python, and explicit network access control lists using Linux firewall utilities.

2 Risk Assessment

2.1 Assets, Vulnerabilities, and Consequences

Asset 1: Student Records Database/Server

Vulnerabilities: Unencrypted file transfers over the local network and guest network routing directly to the server.

Consequences: Interception of sensitive personally identifiable information (PII) via packet sniffing, potential exposure of grade records, and breach of compliance regulations.

Asset 2: Staff User Credentials and Authentication Session

Vulnerabilities: Weak password policy and outdated software/services.

Consequences: Credential stuffing, brute-force attacks, system compromise, and unauthorized administrative privilege escalation.

Asset 3: Internal Network Infrastructure

Vulnerabilities: Unrestricted routing between the guest segment and administrative servers, along with unmonitored inbound external traffic.

Consequences: Denial of Service (DoS) attacks, unauthorized network pivoting, and external exploit execution.

2.2 Risk Rankings and Justification

Risk	Likelihood	Impact	Justification
1. Interception of Unencrypted Data	High	High	Active network exposure via encrypted file transfers and guest routing makes data cap trivially reproducible.
2. Unauthorized Access via Weak Passwords	High	Medium	Passive brute-force attempts f external IPs combined with w credential rules present a probability of compromise.
3. External Unsolicited Server Traffic	Medium	High	Exposure to malicious exte addresses can lead to system nerabilities being exploited.

Table 1: Risk Assessment Matrix

2.3 Recommended Controls

- **Control 1 (Encryption & Integrity Verification):** Deploy AES-256 Symmetric Encryption (CBC mode with PKCS7 padding) for storing and transferring files, combined with SHA-256 cryptographic hashing to ensure data integrity.
- **Control 2 (Network Segmentation & Filtering):** Implement strict stateful firewall rules (`iptables`) separating guest traffic from internal administrative zones, blocking all unauthorized subnet requests.
- **Control 3 (Access Control & Key Management):** Enforce strong password complexity policies, update host software packages, and store system decryption keys outside root code repositories using environment-level key isolation.

3 Encryption and Integrity Application

3.1 Implementation Overview

A custom Python script using the `cryptography` library (`Fernet`/AES-CBC) and `hashlib` was written to manage security functions. The application reads local file binaries, generates cryptographic signatures using SHA-256, encrypts payloads, and handles file exceptions safely.

3.2 Core Cryptographic Implementation

```
1  import os
2  import hashlib
3  from cryptography.fernet import Fernet
4
5  def generate_and_save_key(key_path="secret.key"):
6      key = Fernet.generate_key()
7      with open(key_path, "wb") as key_file:
8          key_file.write(key)
9
10 def load_key(key_path="secret.key"):
11     if not os.path.exists(key_path):
12         raise FileNotFoundError("Encryption key missing!")
13     return open(key_path, "rb").read()
14
15 def encrypt_file(file_path, key):
16     f = Fernet(key)
17     with open(file_path, "rb") as file:
18         file_data = file.read()
19     encrypted_data = f.encrypt(file_data)
20     with open(file_path + ".enc", "wb") as file:
21         file.write(encrypted_data)
22
23 def decrypt_file(enc_path, key):
24     f = Fernet(key)
25     with open(enc_path, "rb") as file:
```

```
        encrypted_data = file.read()
    decrypted_data = f.decrypt(encrypted_data)
    out_path = enc_path.replace(".enc", ".dec")
    with open(out_path, "wb") as file:
        file.write(decrypted_data)

def calculate_sha256(file_path):
    sha256_hash = hashlib.sha256()
    try:
        with open(file_path, "rb") as f:
            for byte_block in iter(lambda: f.read(4096), b""):
                sha256_hash.update(byte_block)
        return sha256_hash.hexdigest()
    except FileNotFoundError:
        print(f"Error: File '{file_path}' not found.")
        return None
```

Listing 1: Python Security Application Excerpt

3.3 Verification Procedures

1. **Encryption Testing:** The plain text student records file (`student_records.txt`) was encrypted into `student_records.txt.enc`. Verification confirmed that the original cleartext was obfuscated.

2. **Decryption Testing:** Decryption resulted in `student_records.txt.dec`. A `diff` check confirmed that the decrypted payload was identical to the input file.

3. **Integrity Verification:** SHA-256 hashes generated before and after file modification demonstrated clear signature drift, correctly alerting the application to payload tampering.

4. **Error Handling:** Validated that attempting to open non-existent paths or passing corrupted decryption keys yielded controlled error warnings rather than crash dumps.

4 Network Traffic Filtering

4.1 Firewall Rule Definitions

Traffic filtering was implemented using `iptables` rules configured on the central Linux record server.

```
# Flush pre-existing chain rules
sudo iptables -F
sudo iptables -X

# Set default policy to drop inbound traffic
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
sudo iptables -P OUTPUT ACCEPT
```

```
9
10 # Allow loopback operations
11 sudo iptables -A INPUT -i lo -j ACCEPT
12 sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j
      ACCEPT
13
14 # Block Guest Subnet Access (192.168.100.0/24) to Records Server
15 sudo iptables -A INPUT -s 192.168.100.0/24 -d 10.0.0.5 -j DROP
16
17 # Permit Authorized Staff Subnet (10.0.10.0/24) Access to HTTP/
      HTTPS Service
18 sudo iptables -A INPUT -p tcp -s 10.0.10.0/24 --dport 443 -j
      ACCEPT
19 sudo iptables -A INPUT -p tcp -s 10.0.10.0/24 --dport 80 -j
      ACCEPT
20
21 # Drop all other unauthorized inbound traffic on destination port
       443/80
22 sudo iptables -A INPUT -p tcp --dport 443 -j DROP
23 sudo iptables -A INPUT -p tcp --dport 80 -j DROP
```

Listing 2: iptables Network Rule Configuration

4.2 Filter Verification Test Execution

Verification scenarios were evaluated using netcat (`nc`) connectivity probes.

Test Connection	Command Executed	Expected Outcome	Actual Result
1. Staff Subnet Access (Permitted)	`nc -zv -w 3 10.0.0.5 443` (from 10.0.10.15)	Connection Succeeded	Connected to 10.0.0.5 port 443 [tcp/https] open
2. Guest Subnet Access (Blocked)	`nc -zv -w 3 10.0.0.5 443` (from 192.168.100.22)	Connection Timed Out / Dropped	nc: connect to 10.0.0.5 port 443 (tcp) timed out
3. Unfamiliar External Host (Blocked)	`nc -zv -w 3 10.0.0.5 443` (from 203.0.113.50)	Connection Timed Out / Dropped	nc: connect to 10.0.0.5 port 443 (tcp) timed out

Table 2: Firewall Filter Verification Results

5 Project Evidence and GitHub Repository Submission

All source artifacts, environment setup documentation, test cases, and repository scripts have been pushed to GitHub.

5.1 Repository Layout

The repository structure is arranged as follows:

```
crypto_exam/
 README.md
 risk_assessment.md
 filter_tests.md
 app/
    security_toolkit.py
    requirements.txt
 firewall/
    rules.sh
 docs/
     report.tex
     report.pdf
```

6 Conclusion

This project addressed the key vulnerabilities identified in the initial security review. By deploying Fernet AES-256 binary encryption and SHA-256 checksum monitoring, data in transit and at rest remains secure against eavesdropping and tampering. Restricting network boundaries via `iptables` prevents unauthorized guest access and external probing.

The complete codebase, configurations, and documentation can be cloned from: `https://github.com/Peace-btech/crypto_exam`